

Fattoriale e Fibonacci

Successioni con i cicli, e quando i numeri interi non bastano

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027



SAPIENZA
UNIVERSITÀ DI ROMA

Se so quante volte: for

Quando il numero di passi è fissato: la tabella dei quadrati da 1 a n, con while e con for

quadrati.c (con while)

```c
int main() {
    int n, i;
    printf("n = ");
    scanf("%d", &n);
    i = 1;
    while (i <= n) {
        printf("%d\t%d\n", i, i * i);
        i++;
    }
}
```

quadrati.c (con for)

```c
int main() {
    int n, i;
    printf("n = ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        printf("%d\t%d\n", i, i * i);
    }
}
```

i = 1

inizio: si esegue una volta sola, prima del ciclo

i <= n

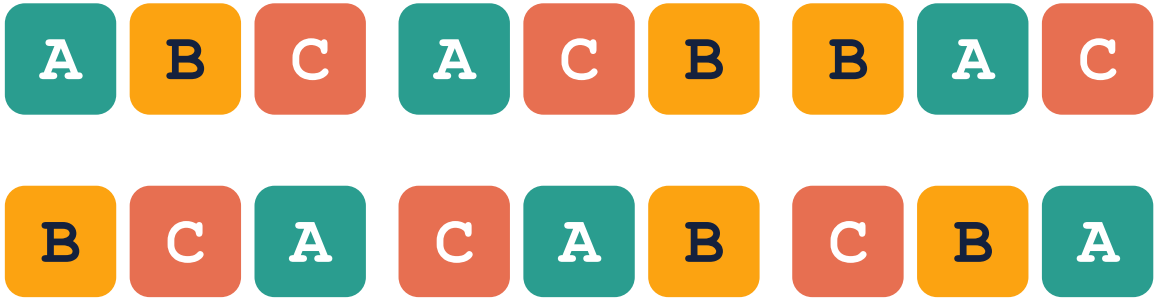
condizione: si controlla prima di ogni giro

i++

passo: si esegue alla fine di ogni giro

In quanti modi si ordinano n libri?

Con tre libri A, B, C i modi sono 6: serve il fattoriale n!



$n! = n \cdot (n-1) \cdot \ldots \cdot 2 \cdot 1$

il primo libro si sceglie tra n, il secondo tra n − 1, e così via

$0! = 1$

per definizione: un solo modo di ordinare nessun libro

n	n! $(n \cdot (n-1)!)$
1	1
2	2 · 1 = 2
3	3 · 2 = 6
4	4 · 6 = 24
5	5 · 24 = 120

Il fattoriale con un ciclo for

fattoriale.c, prima versione: fatt parte da 1 e si moltiplica per i a ogni giro

fattoriale.c (con int)

```c
int main() {
    int n, i;
    int fatt = 1;
    printf("n = ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        fatt *= i;
    }
    printf("%d! = %d\n", n, fatt);
}
```

Terminale

```
$ ./app.exe
n = 5
5! = 120
$ ./app.exe
n = 10
10! = 3628800
$ ./app.exe
n = 15
15! = 2004310016
```

int fatt = 1;

parte da 1, non da 0: il prodotto vuoto

fatt *= i;

come fatt = fatt * i, a ogni giro

15! = 2004310016

sbagliato: un int arriva a 2147483647, senza avvisi

Più spazio: unsigned long long

fattoriale.c: 64 bit senza segno, stampati con %llu; n non negativo con do/while

fattoriale.c

```c
int main() {
    int n, i;
    unsigned long long fatt = 1;
    do {
        printf("n (>= 0): ");
        scanf("%d", &n);
    } while (n < 0);
    for (i = 1; i <= n; i++) {
        fatt *= i;
    }
    printf("%d! = %llu\n", n, fatt);
}
```

Terminale

```
$ ./app.exe
n (>= 0): 0
0! = 1
$ ./app.exe
n (>= 0): 20
20! = 2432902008176640000
$ ./app.exe
n (>= 0): 21
21! = 14197454024290336768
```

unsigned long long

64 bit senza segno: da 0 a 18446744073709551615

%llu

il formato per stamparlo: l, l, u

do ... while (n < 0)

come in M04; con 0 il for non gira: 0! = 1

Perché 21! è sbagliato

Il massimo di un unsigned long long è $2^{64} - 1$, circa $1.8 \cdot 10^{19}$

	valore	entra in 64 bit?
`20!`	`2432902008176640000`	sì: $2.4 \cdot 10^{18}$
`21!`	`51090942171709440000`	no: $5.1 \cdot 10^{19} > 1.8 \cdot 10^{19}$
`massimo`	`18446744073709551615`	$2^{64} - 1$

`Il programma stampa 14197454024290336768` per 21!: è il resto della divisione per 2^{64}. Sembra un numero normale, e non c'è nessun errore né avviso.

Regola pratica

controllare sempre che il risultato entri nel tipo

Due strade

tipo più grande, oppure un double (slide dopo)

Numeri enormi: il double

fattoriale_double.c: meno cifre esatte, ma arriva fino a 170!

fattoriale_double.c

```c
int main() {
    int n, i;
    double fatt = 1.;
    do {
        printf("n (>= 0): ");
        scanf("%d", &n);
    } while (n < 0);
    for (i = 1; i <= n; i++) {
        fatt *= i;
    }
    printf("%d! = %e\n", n, fatt);
    printf("%d! = %g\n", n, fatt);
}
```

Terminale

```
$ ./app.exe
n (>= 0): 20
20! = 2.432902e+18
20! = 2.4329e+18
$ ./app.exe
n (>= 0): 170
170! = 7.257416e+306
170! = 7.25742e+306
$ ./app.exe
n (>= 0): 171
171! = inf
171! = inf
```

%e %g

notazione scientifica: i formati della prima ora

170!

7.257e+306: vicino al massimo di un double

inf

171! supera il massimo ($1.8 \cdot 10^{308}$): infinito

Ogni numero è la somma dei due precedenti

Successione di Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, … $F(n) = F(n-1) + F(n-2)$

0	1	1	2	3	5	8	13
F0	F1	F2	F3	F4	F5	F6	F7

a0	a1	an
0	1	?
F(0)	F(1)	il prossimo

un passo

```
an = a0 + a1;
a0 = a1;
a1 = an;
```

Due variabili d'appoggio

per ricordare gli ultimi due termini: a ogni passo si calcola an e poi si scorre tutto di un posto

Cosa succede a ogni giro

Traccia a mano di fibonacci.c con n = 7: a0 = 0, a1 = 1 all'inizio

giro i	an = a0 + a1	a0 dopo	a1 dopo (= an)
2	0 + 1 = 1	1	1
3	1 + 1 = 2	1	2
4	1 + 2 = 3	2	3
5	2 + 3 = 5	3	5
6	3 + 5 = 8	5	8
7	5 + 8 = 13	8	13
8	i <= 7 => 0		esce dal ciclo

Il programma di Fibonacci

fibonacci.c: un ciclo for con le due variabili d'appoggio, per n tra 1 e 93

fibonacci.c

```c
int main() {
    int n, i;
    unsigned long long a0 = 0, a1 = 1, an;   // F(0) e F(1)
    do {
        printf("n (1-93): ");
        scanf("%d", &n);
    } while (n < 1 || n > 93);
    printf("F(1) = %llu\n", a1);
    for (i = 2; i <= n; i++) {
        an = a0 + a1;
        a0 = a1;
        a1 = an;
        printf("F(%d) = %llu\n", i, an);
    }
}
```

Terminale

```
$ ./app.exe
n (1-93): 7
F(1) = 1
F(2) = 1
F(3) = 2
F(4) = 3
F(5) = 5
F(6) = 8
F(7) = 13
```

Terminale (94 rifiutato, poi 93)

```
$ ./app.exe
n (1-93): 94
n (1-93): 93
F(93) = 12200160415121876738
```

`F(93) = 12200160415121876738` è l'ultimo che entra in 64 bit: per questo n arriva al massimo a 93.

Il rapporto tra termini consecutivi

rapporto.c: F(n + 1) / F(n) si avvicina a un numero preciso

rapporto.c

```c
#include <math.h>
#include <stdio.h>

int main() {
    int i;
    unsigned long long a0 = 1, a1 = 1, an;    // F(1) e F(2)
    printf(" n\tF(n+1)/F(n)\n");
    for (i = 1; i <= 15; i++) {
        printf("%2d\t%.10f\n", i, (double) a1 / a0);
        an = a0 + a1;
        a0 = a1;
        a1 = an;
    }
    printf("phi\t%.10f\n", (1. + sqrt(5.)) / 2.);
}
```

`(double) a1 / a0`: cast, senza la divisione sarebbe intera

Terminale

```
$ ./app.exe
 n      F(n+1)/F(n)
 1      1.0000000000
 2      2.0000000000
 3      1.5000000000
 4      1.6666666667
 5      1.6000000000
 6      1.6250000000
 7      1.6153846154
 8      1.6190476190
 9      1.6176470588
10      1.6181818182
11      1.6179775281
12      1.6180555556
13      1.6180257511
14      1.6180371353
15      1.6180327869
phi     1.6180339887
```

Esercizi

Da fare a casa, poi si controlla con un programma in C

1 Combinazioni

Calcola in quanti modi si scelgono k oggetti tra n: n! / (k! (n − k)!), con tre cicli for e variabili double. Stampa con %.0f.

prova con:

```
n k
10 3
49 6
90 5
```

2 Numeri di Lucas

Come Fibonacci, ma parte da 2 e 1: 2, 1, 3, 4, 7, … Leggi n e stampa i primi n termini, uno per riga.

prova con:

```
10
15
```

3 Limite di int

Trova il più grande n per cui n! entra in un int (massimo 2147483647), con un while. Stampa n e n!.

prova con:

nessun input

Compila con `gcc -Wall -Wextra -o app.exe nome.c -lm` (-lm serve solo con math.h)

Riepilogo

1	**Fattoriale**	n! con un for; 0! = 1; int, unsigned long long (%llu) e double (%e): ogni tipo ha il suo limite
2	**Overflow**	oltre il massimo il risultato è sbagliato senza avvisi (21! in 64 bit, 15! in int); il double dà inf oltre 170!
3	**Fibonacci**	due variabili d'appoggio e un ciclo; F(93) è l'ultimo in 64 bit; il rapporto tende alla sezione aurea